Cours — Fonctions et boucles en Python

1re NSI

1. Les fonctions

1.1 Principe

Définition Une **fonction** est un bloc d'instructions auquel on donne un **nom**. On la **définit** une seule fois, puis on l'**appelle** autant de fois que nécessaire. Elle peut recevoir des valeurs en entrée (les **paramètres**) et **renvoyer** un résultat (avec `return`).



exemple : `5` ⟶ `double` ⟶ `10`

Intérêt : éviter de répéter du code, découper un problème en petites tâches, rendre le programme plus lisible et plus facile à tester et à corriger.

Syntaxe :

```
def nom_fonction(param1, param2):     ← en-tête : mot-clé def, nom, paramètres entre parenthèses, deux-points :
    instructions                      ← corps de la fonction, indenté (4 espaces)
    return resultat                   ← renvoie le résultat et termine la fonction
```

Vocabulaire :

- **définir** une fonction : écrire le bloc `def ...` (rien ne s'exécute encore) ;
- **appeler** une fonction : écrire son nom suivi de parenthèses, par exemple `double(5)` ;
- **paramètre** : variable écrite dans la définition (`x` dans `def double(x):`) ;
- **argument** : valeur donnée lors de l'appel (`5` dans `double(5)`).

1.2 Fonction sans paramètre

Les parenthèses restent obligatoires, à la définition comme à l'appel.

▸ script.py

```
def figure_petit_chat():
    print(" ^----^")
    print("  _00_")
    print(" |-==-|")
```

▸ Shell Python

```
>>> figure_petit_chat()
 ^----^
  _00_
 |-==-|
```

`print` ou `return` ?

▸ script.py

```
def saluer():
    print("Bonjour")      # affiche

def message():
    return "Bonjour"      # renvoie
```

▸ Shell Python

```
>>> saluer()
Bonjour
>>> m = message()
>>> m + " !"
'Bonjour !'
>>> s = saluer()
Bonjour
>>> print(s)
None   rien n'a été renvoyé
```

À retenir `print` **affiche** une valeur à l'écran, mais on ne peut plus s'en servir. `return` **renvoie** une valeur que l'on peut stocker dans une variable ou réutiliser dans un calcul. Dans une fonction, on utilise en général `return` plutôt que `print`.

1.3 Fonction avec paramètre(s)

Lors de l'appel, chaque **argument** est affecté au **paramètre** correspondant, **dans l'ordre**.

▸ script.py

```
def salut(nom):
    return "bonjour " + nom

def diviser(a, b):
    return a / b

def perimetre_rectangle(longueur, largeur):
    return 2 * (longueur + largeur)
```

▸ Shell Python

```
>>> salut("John")
'bonjour John'
>>> diviser(10, 2)
5.0   a = 10, b = 2
>>> diviser(2, 10)
0.2   l'ordre compte !
>>> perimetre_rectangle(5, 3)
16
```

Une fonction peut en **appeler une autre** :

▸ script.py

```
def carre(x):
    return x * x

def somme_carres(a, b):
    return carre(a) + carre(b)
```

▸ Shell Python

```
>>> carre(3)
9
>>> somme_carres(3, 4)
25
```

1.4 Importer une fonction ou un module

Un **module** est un fichier qui contient des fonctions déjà écrites : `math` (mathématiques), `random` (hasard), `turtle` (dessin)… Il faut l'**importer** avant d'utiliser ses fonctions.

Importation	Utilisation	Remarque
`import math`	`math.sqrt(16)`	on écrit le nom du module devant la fonction
`from math import sqrt, pi`	`sqrt(16)`	on n'importe que les fonctions utiles
`from math import *`	`sqrt(16)`	importe tout le module (à éviter : risque de conflit de noms)

▸ script.py

```
from math import pi, degrees, radians

def cube(x):
    return x ** 3

def volume_sphere(r):
    return 4 / 3 * pi * cube(r)
```

▸ Shell Python

```
>>> volume_sphere(5)
523.5987755982989
>>> degrees(3.14)
179.9087476710785
>>> radians(45)
0.7853981633974483
>>> help(radians)
affiche la documentation de la fonction
```

Activités → Notebook **TP5a** (fonctions), puis **Partie A** de la fiche d'exercices.

2. Les boucles

2.1 Principe

Une **boucle** permet de **répéter** un bloc d'instructions (le **corps** de la boucle, indenté). Python propose deux types de boucles :

Boucle bornée : `for`	Boucle non bornée : `while`
Le nombre de répétitions est **connu à l'avance** : on parcourt une séquence de valeurs.	On répète **tant qu'une condition est vraie** : le nombre de répétitions n'est pas connu à l'avance.

2.2 La boucle bornée `for`

```
for variable in iterable:      ← la variable prend successivement chaque valeur de l'itérable
    instructions               ← corps de la boucle, indenté, exécuté une fois par valeur
```

La variable de boucle s'appelle le **variant**. L'**itérable** peut être un `range`, une chaîne de caractères ou une liste :

Instruction	Valeurs prises par `i`	
`for i in range(4):`	`0, 1, 2, 3`	4 valeurs, de 0 à 3 : la borne de fin est **exclue**
`for i in range(1, 4):`	`1, 2, 3`	de 1 (inclus) à 4 (exclu)
`for i in range(0, 10, 2):`	`0, 2, 4, 6, 8`	le 3e nombre est le **pas**
`for i in "TGV":`	`'T', 'G', 'V'`	chaque caractère de la chaîne
`for i in [6, 4, 2]:`	`6, 4, 2`	chaque élément de la liste

▸ script.py

```
for i in range(1, 4):
    print(i, "x 7 =", i * 7)
```

▸ Shell Python

```
>>> (exécution de script.py)
1 x 7 = 7
2 x 7 = 14
3 x 7 = 21
```

Boucle dans une fonction : le compteur. On initialise une variable **avant** la boucle, puis on la met à jour **dans** la boucle.

▸ script.py

```
def compter(lettre, mot):
    n = 0                      # compteur
    for c in mot:
        if c == lettre:
            n = n + 1
    return n                   # après la boucle
```

▸ Shell Python

```
>>> compter("a", "abacab")
3
>>> compter("z", "abacab")
0
```

2.3 La boucle non bornée `while`

```
initialisation          ← on donne une valeur de départ à la variable testée
while condition:        ← condition d'exécution : on entre dans la boucle tant qu'elle est vraie
    instructions        ← corps de la boucle, indenté
    mise à jour         ← modifie la variable testée, pour que la condition devienne fausse
```

La boucle s'arrête quand la **condition d'arrêt** (le contraire de la condition d'exécution) est vraie :

Condition d'exécution	Condition d'arrêt	
`while i < 4:`	`i >= 4`	
`while i != 0:`	`i == 0`	
`while a > 0 and b > 0:`	`a <= 0 or b <= 0`	`and` devient `or`

Exemple : quelle est la première puissance de 2 supérieure ou égale à 100 ?

▸ script.py

```
n = 1
while n < 100:
    n = n * 2
print(n)
```

▸ Shell Python

```
>>> (exécution)
128
```

Tableau de suivi (trace) :

n	1	2	4	8	16	32	64	128
n < 100	V	V	V	V	V	V	V	**F**

7 tours de boucle ; on sort quand `n` vaut 128.

Équivalence `for` / `while` : les deux scripts suivants affichent les mêmes valeurs.

▸ avec for

```
for i in range(10):
    print(i * 2)
```

▸ avec while

```
i = 0                # initialisation
while i < 10:
    print(i * 2)
    i = i + 1        # mise à jour
```

À retenir Si on oublie la **mise à jour** (`i = i + 1`), la condition reste toujours vraie : c'est une **boucle infinie**. Le programme ne s'arrête jamais (il faut l'interrompre : `Ctrl+C` ou bouton *Stop* de Pyzo).

2.4 Terminaison et variant de boucle

Définition Un **variant de boucle** est une quantité **entière**, **positive**, qui **diminue strictement** à chaque tour de boucle. Si une boucle `while` possède un variant, elle **se termine** forcément.

▸ script.py

```
def compte_a_rebours(n):
    while n > 0:
        print(n)
        n = n - 1
    print("Décollage !")
```

▸ Shell Python

```
>>> compte_a_rebours(3)
3
2
1
Décollage !
```

Ici, `n` est un **variant** : c'est un entier positif qui diminue de 1 à chaque tour. Il finit par valoir 0, la condition `n > 0` devient fausse et la boucle **se termine** (après `n` tours).

À retenir

- On utilise `for` quand on connaît le nombre de répétitions, ou pour **parcourir** une chaîne ou une liste.
- On utilise `while` quand on répète **tant qu'**une condition est vraie.
- Avec `while`, il faut toujours vérifier que la condition finira par devenir fausse (trouver un variant).

Activités → **Parties B et C** de la fiche d'exercices.